

天工行者机器人：示教器生成到真机部署

说明

- 参考框架: ROS 2 Humble + Python GUI
- 适配机型: 天工无疆（Pro）
- 操作系统: Ubuntu 22.04
- 依赖环境: ROS 2 Humble、Python 3.10+

一、环境要求

项目	要求
操作系统	Ubuntu 22.04
ROS 2 版本	Humble
Python	3.10+
ROS 2 软件包	ros-humble-desktop、colcon-common-extensions、robot-state-publisher、joint-state-publisher、xacro、rviz2
Python 依赖	python3-tk、inputs、pynput（用于 sim_joy 遥控器监听）
外部消息包	lyre_msgs（音频控制）、bodyctrl_msgs（关节控制） - 已包含在 other_install/ 目录中

二、项目结构

代码块

```
1  motion_builder/  
2  |— run_create.sh           # 启动时间轴编辑器  
3  |— run_editor.sh          # 启动动作精修工具  
4  |— run_sim.sh             # 启动仿真播放环境  
5  |— run_real.sh            # 启动真实机播放环境  
6  |— save/                  # 动作文件存储目录
```

```
7 | └─ src/
8 |   └─ tienkung_action/          # 动作播放包
9 |     └─ config/
10 |       └─ scenarios.json        # 场景配置文件
11 |       └─ scripts/
12 |         └─ extract_arm_action.py # 动作提取工具
13 |     └─ tiangong2pro_urdf/      # 机器人URDF模型包
14 |       └─ launch/
15 |         └─ editor.launch.py    # 时间轴编辑器启动文件（被run_create.sh调用）
16 |         └─ interactive_gui.launch.py # Pose创建器启动文件（被run_editor.sh调用）
17 |       └─ simulation.launch.py  # 仿真环境启动文件
18 |   └─ sim_joy/                  # 遥控器手柄控制包
19 | └─ install/                    # 编译输出目录
20 |   └─ tienkung_action/
21 |     └─ share/
22 |       └─ tienkung_action/
23 |         └─ config/
24 |           └─ scenarios.json    # 运行时使用的场景配置
25 |           └─ actions/         # 动作JSON文件目录
26 |             └─ editor/        # 动作编辑器保存目录
27 |             └─ create/        # 动作录制器保存目录
28 | └─ other_install/             # 外部依赖消息包
29 |   └─ lyre_msgs/               # 音频播放控制消息
30 |   └─ bodyctrl_msgs/           # 机器人关节控制消息
```

三、快速开始

启动命令

功能	命令	说明
Pose创建器	<code>./run_create.sh</code>	通过关节数值创建新动作
动作精修	<code>./run_editor.sh</code>	对已有动作进行精细调整，检查动作超速
仿真播放	<code>./run_sim.sh</code>	在仿真环境中播放动作
真实机播放	<code>./run_real.sh</code>	在真实机器人上播放动作

使用前提

1. 确保 ROS 2 Humble 已安装并 source:

代码块

```
1 source /opt/ros/humble/setup.bash
```

2. 脚本会自动完成以下操作:

- Source 依赖消息包 (lyre_msgs, bodyctrl_msgs)
- 编译相关包
- Source 工作空间
- 启动对应的 GUI 或节点

3. 直接运行即可，无需额外配置:

代码块

```
1 cd ~/motion_builder
2 ./run_create.sh      # 启动Pose创建器
3 ./run_editor.sh      # 启动动作精修工具，检查动作超速
4 ./run_sim.sh         # 启动仿真播放
5 ./run_real.sh        # 启动真实机播放
```

四、运行脚本详细说明

4.1 ./run_create.sh - Pose创建器（通过关节数值创建动作）

功能: 通过关节数值调节控制机器人，添加关键帧(pose)和时间，生成动作文件。

启动内容:

- `robot_state_publisher` - 机器人状态发布
- `sim_joint_bridge` - 仿真关节桥接
- `action_editor.py` - 时间轴编辑GUI
- `rviz2` - 3D可视化界面

文件保存位置:

代码块

```
1 install/tienkung_action/share/tienkung_action/config/actions/create/
```

```

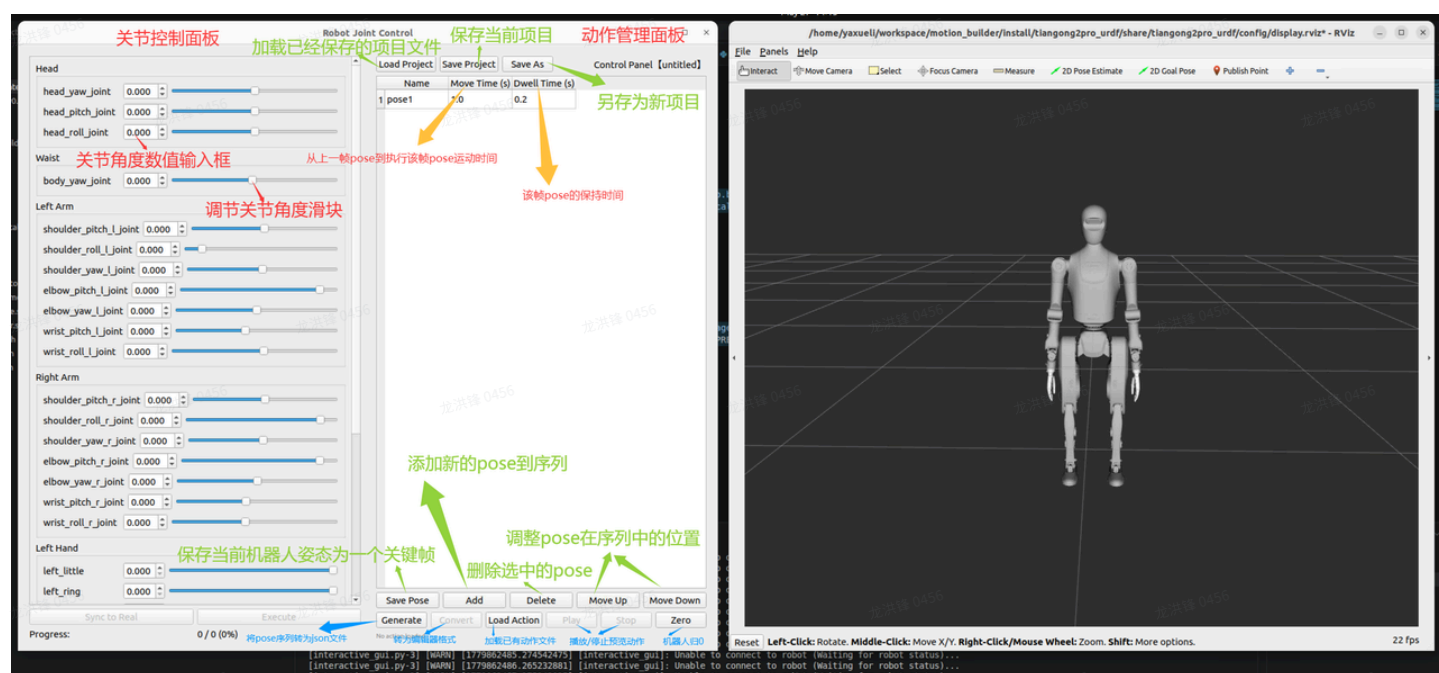
2  └─ {项目名}/
3    └─ {项目名}.json      # Pose设定文件（保存各个关键帧的关节数据）
4    └─ action.json        # 动作文件（Generate生成，用于播放）

```

每个项目保存为一个独立的子目录，包含：

- **{项目名}.json** - 通过"Save Project"保存的项目文件，记录所有pose的关节数据和时间设置
- **action.json** - 点击"Generate"按钮生成的动作文件，可直接用于仿真和真机播放

GUI界面详细说明:



左侧关节控制面板:

- 每个关节有滑块(slider)和数值输入框(spinbox)
- 实时调节关节角度，机器人即时响应
- 支持关节：头部、手臂、灵巧手

右侧动作管理面板:

- **Load Project** - 加载已保存的项目文件（包含所有pose）
- **Save Project** - 保存当前项目
- **Save As** - 另存为新项目
- **Save Pose** - 保存当前机器人姿态为一个关键帧
- **Add** - 添加新的pose到序列
- **Delete** - 删除选中的pose
- **Move Up/Down** - 调整pose在序列中的顺序

- **Generate** - 将pose序列转换为动作JSON文件

- **Convert** - 转换为编辑器格式，保存到

`install/tienkung\_action/share/tienkung\_action/config/actions/editor/` 目录

- **Load Action** - 加载已有动作文件

- **Play/Stop** - 播放/停止预览动作

- **Zero** - 机器人归零

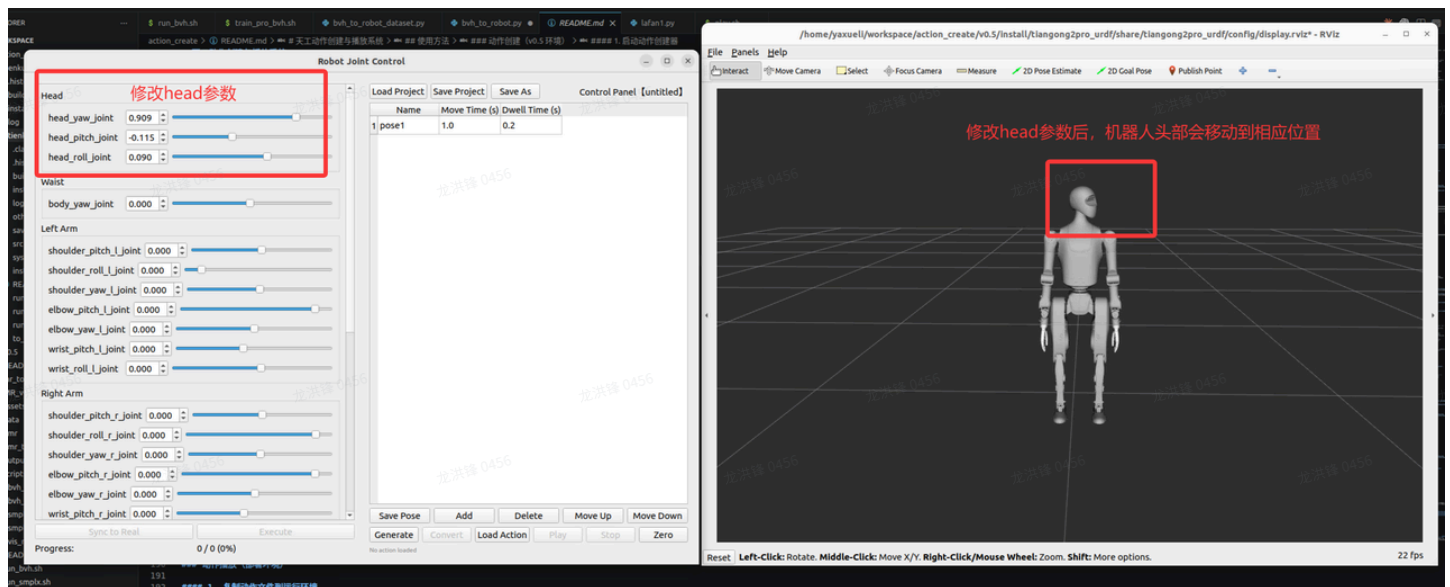
时间控制:

- Move时间: pose之间的运动时间
- Dwell时间: pose保持时间

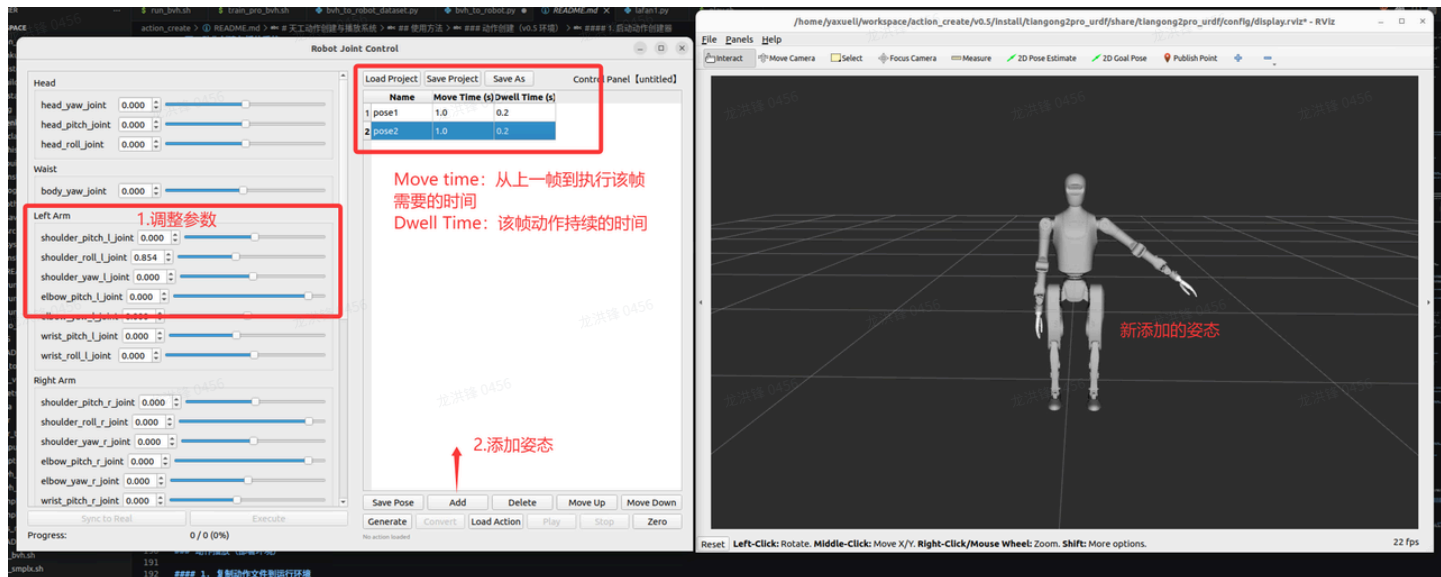
手指编辑技巧: 编辑手指动作时，建议先调整四指（食指、中指、无名指、小指），最后再收拢大拇指，这样可以避免手指相互卡住。

例如:

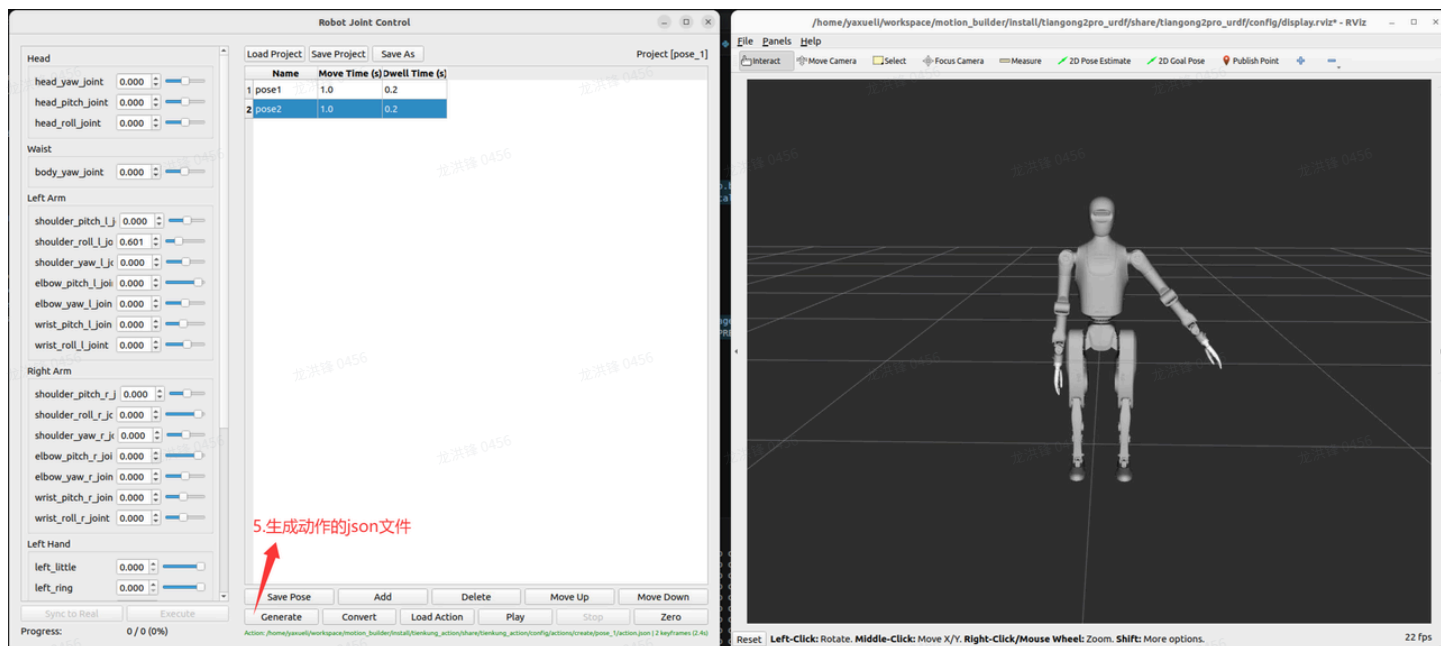
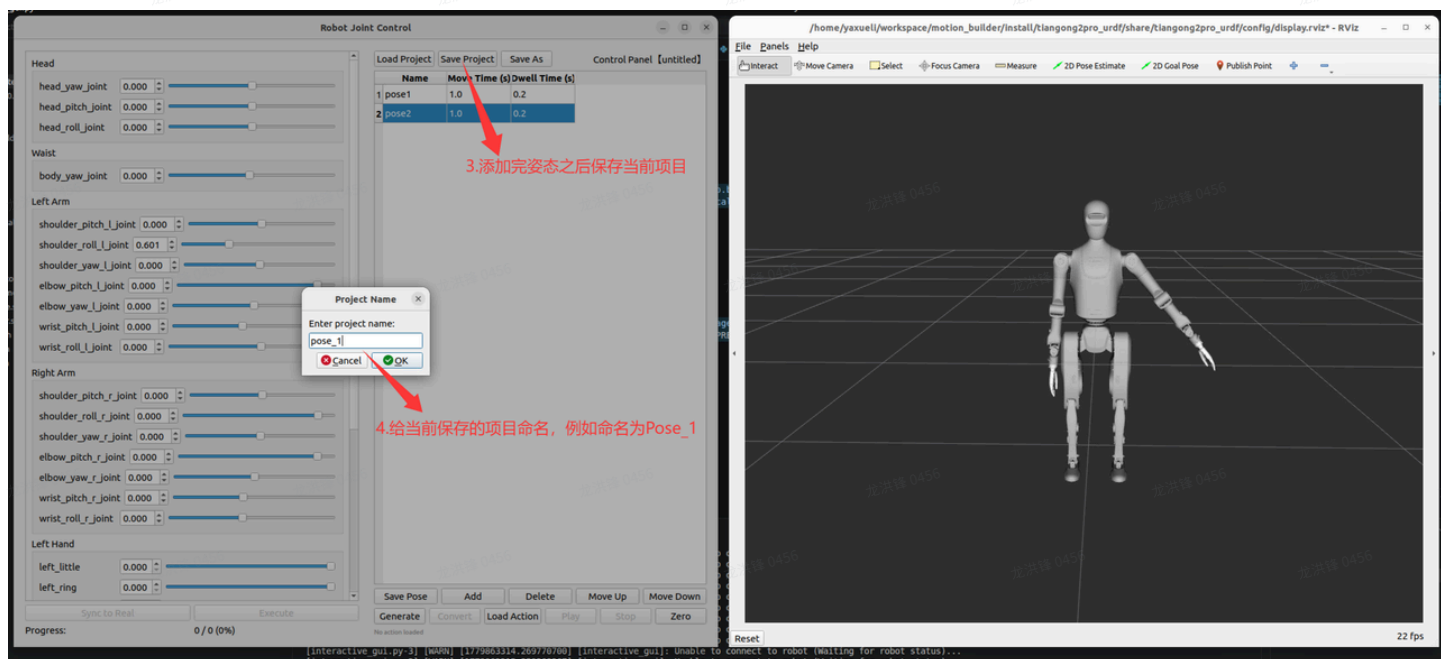
1. 调整head参数改变机器人头部姿态

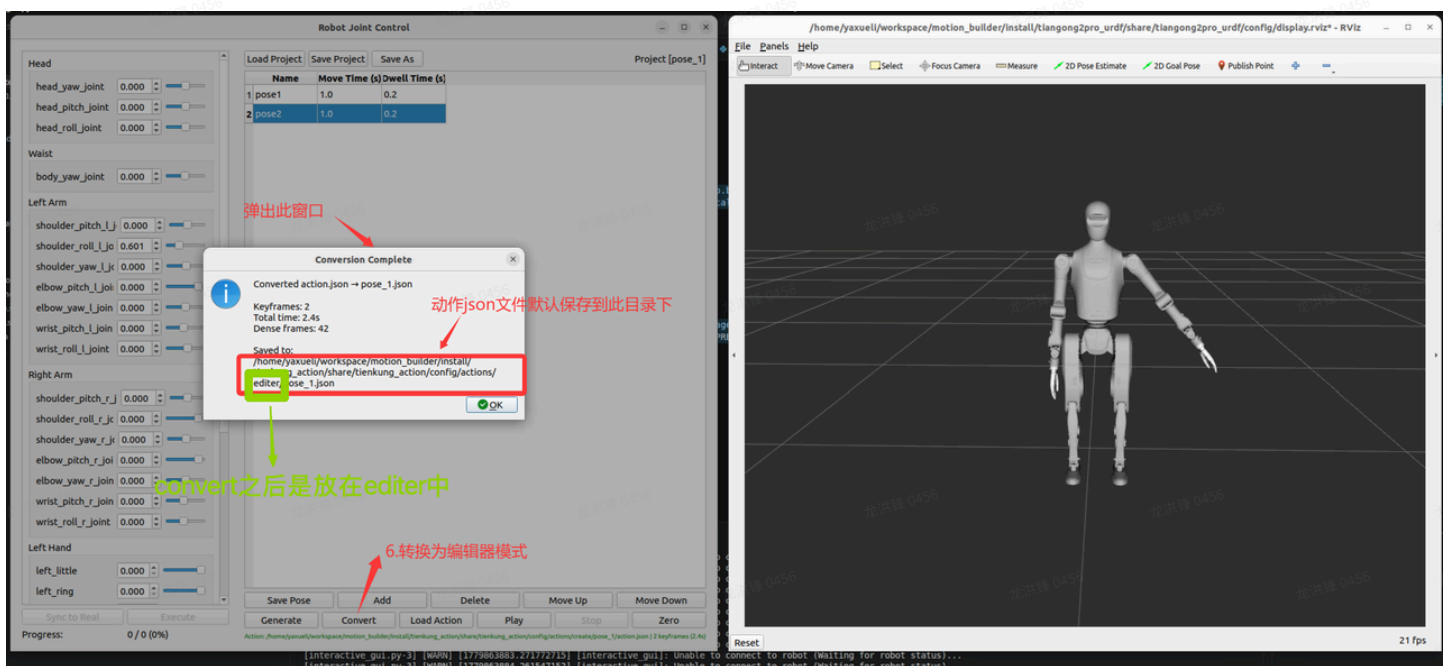
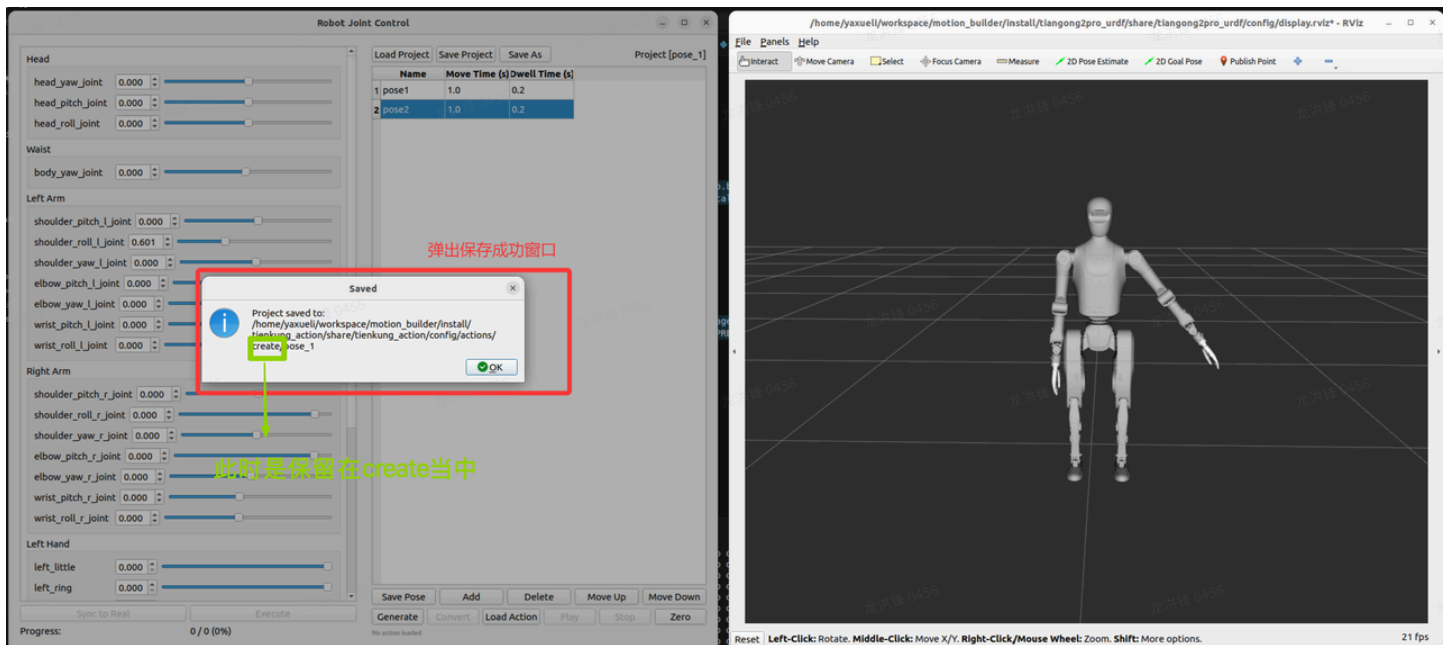


2. 添加机器人姿态

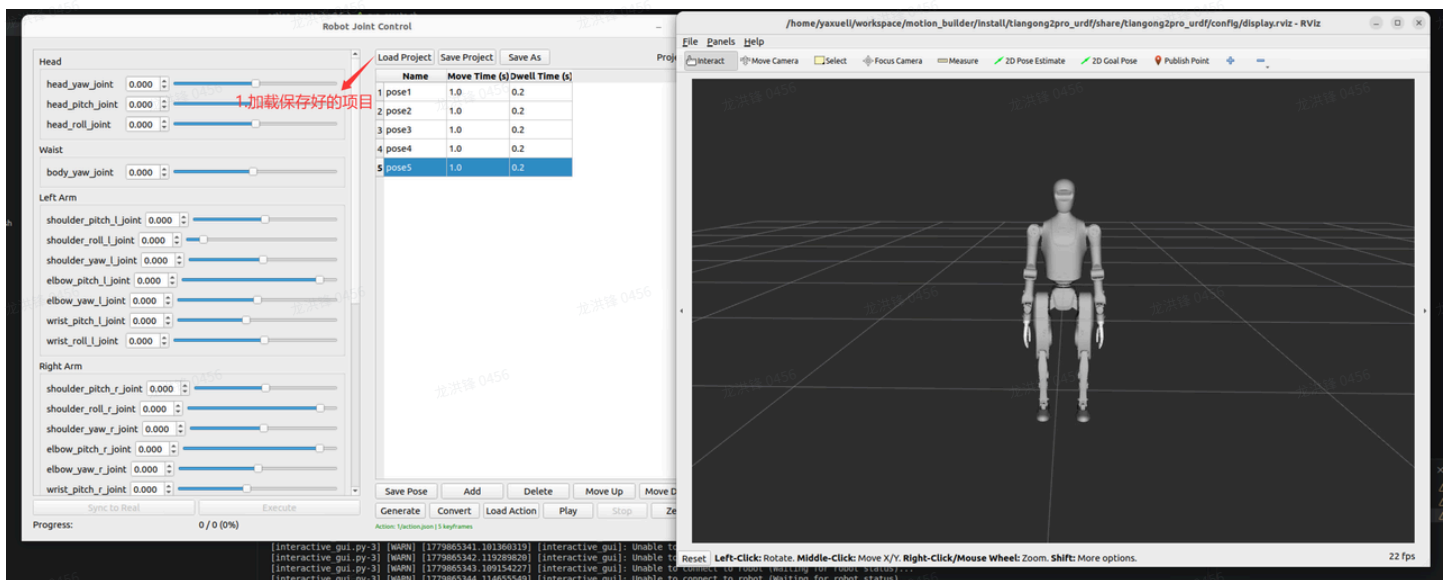


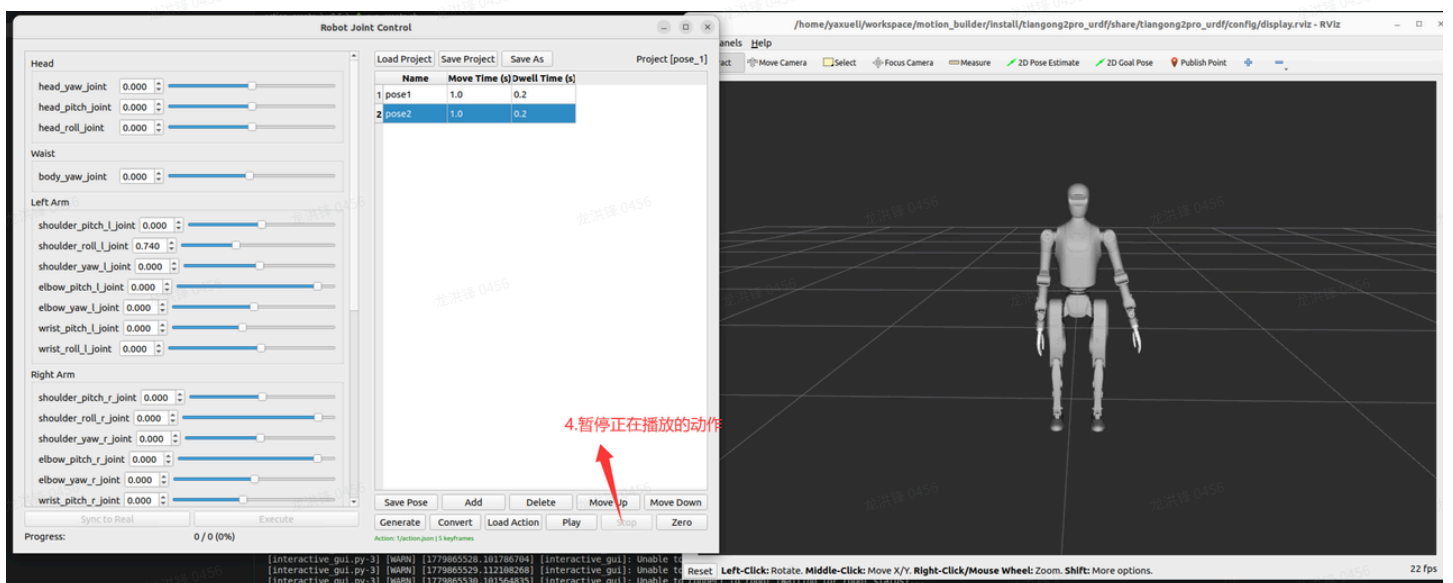
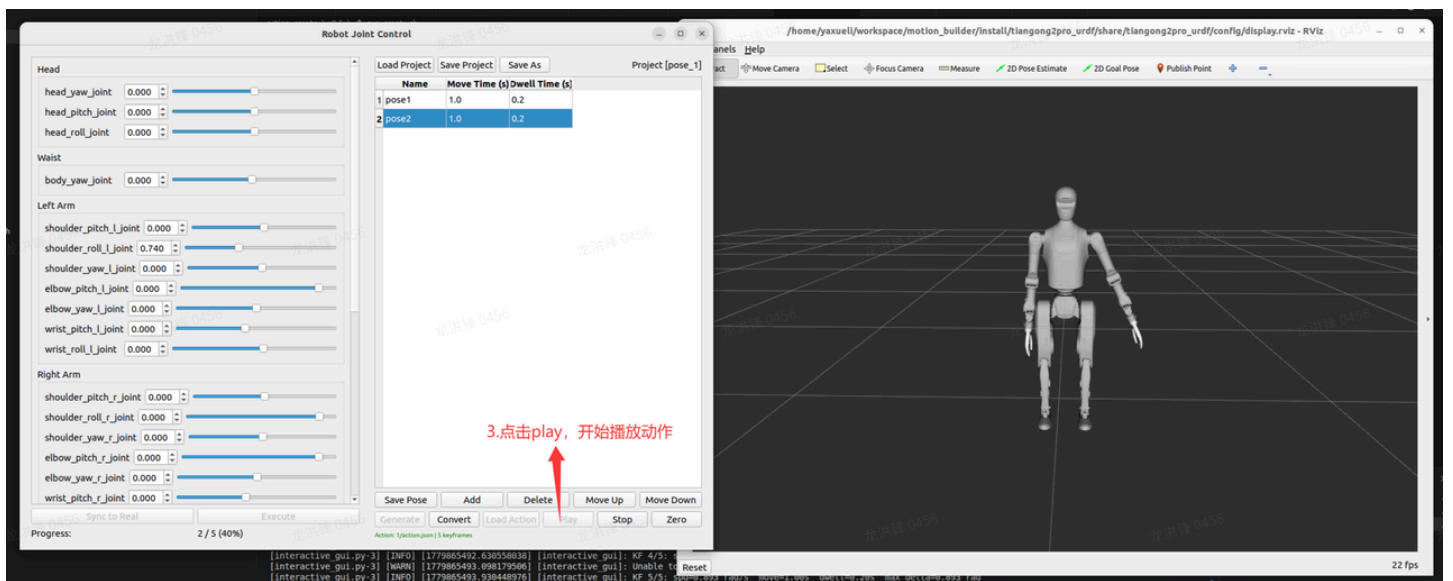
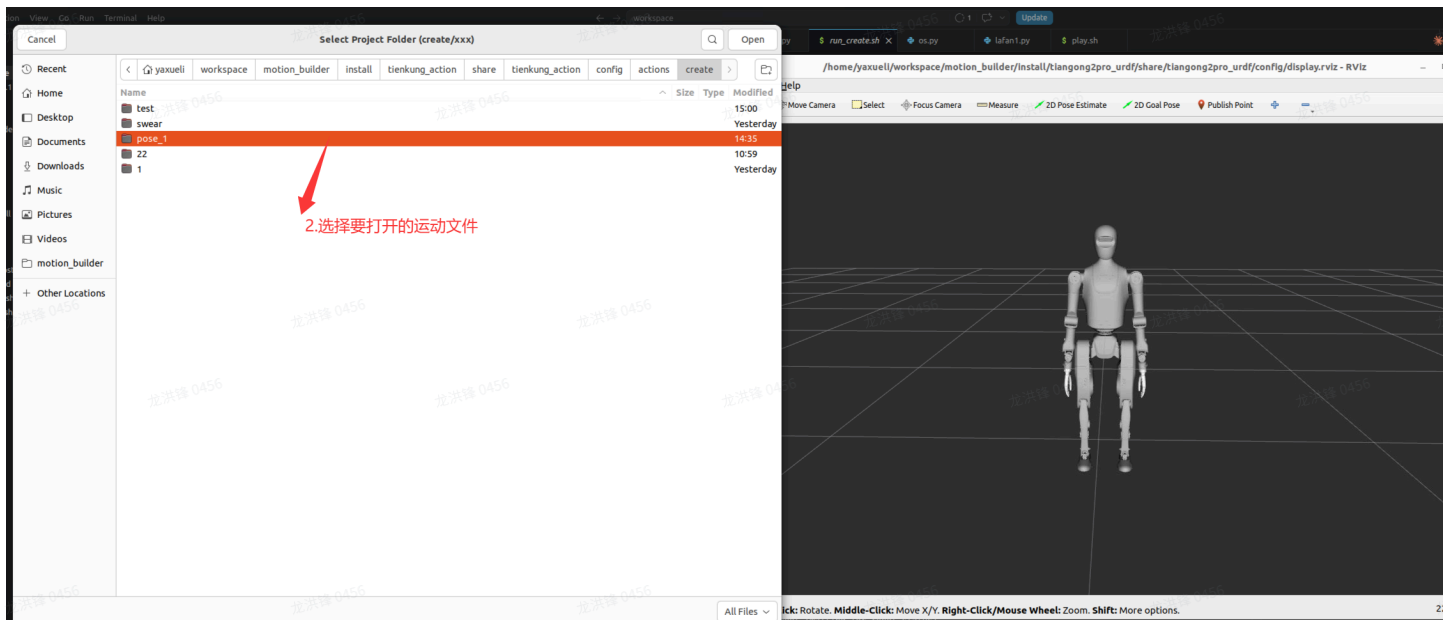
3. 保存添加好的机器人姿态文件





4.加载已有的动作并播放，暂停





4.2 ./run_editor.sh - 动作精修工具（对已创建的动作进行精细调整，检查动作超速）

功能: 加载在 create 中创建的动作文件，对特定帧范围进行精细调整和修正。

启动内容:

- `robot_state_publisher` - 机器人状态发布
- `sim_joint_bridge` - 仿真关节桥接
- `action_editor.py` - 时间轴编辑GUI
- `rviz2` - 3D可视化界面

文件位置:

代码块

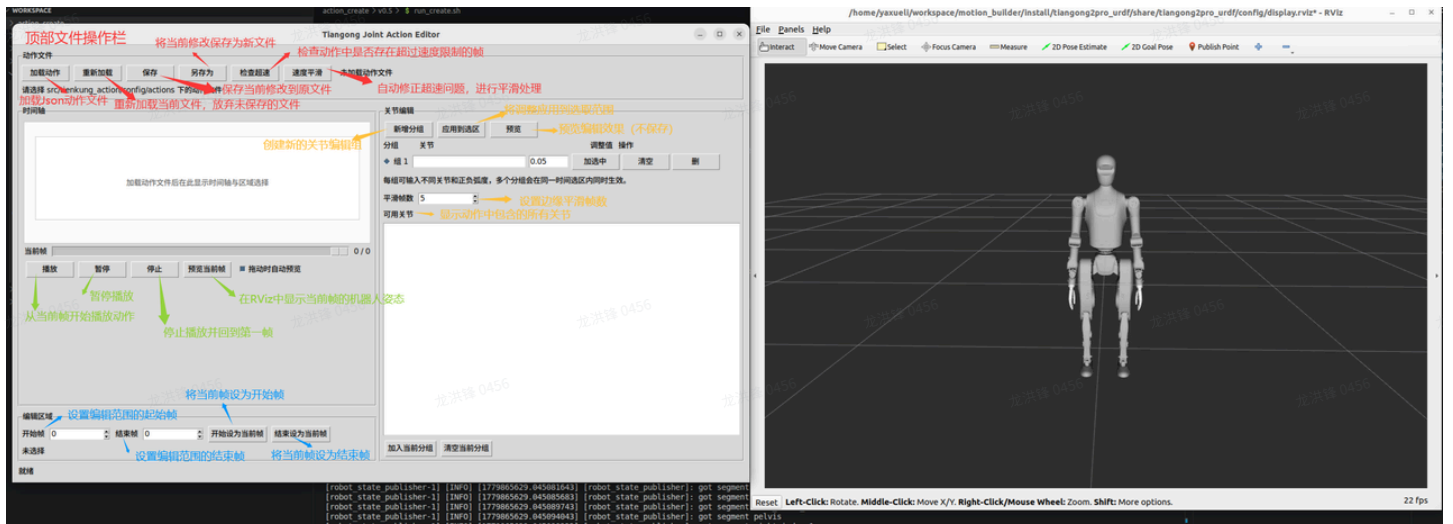
```
1 # 加载文件地址:
2 install/tienkung_action/share/tienkung_action/config/actions/editor/{动作名}.json
3
4 # 保存文件地址:
5 install/tienkung_action/share/tienkung_action/config/actions/{动作名}.json
```

检测动作是否超速，可以通过速度平滑按钮解决部分超速问题，直接保存精修后的动作JSON文件，可直接用于仿真和真机播放。

> 注意:

- 核心功能是精修动作和检查超速，动作编辑之后一定要检查超速问题
- 完成精修或排除超速问题之后，需要确保点击保存按钮，不然就不能将修改的动作文件放到对应的文件下

GUI界面详细说明:



顶部文件操作栏:

- **加载动作** - 打开文件选择对话框，加载JSON动作文件
- **重新加载** - 重新加载当前文件，放弃未保存的修改
- **保存** - 保存当前修改到原文件
- **另存为** - 将当前修改保存为新文件
- **检查超速** - 检查动作中是否存在超过速度限制的帧
- **速度平滑** - 自动修正超速问题，进行平滑处理

时间轴区域:

- 可视化显示动作的时间轴和所有帧
- 拖动选择帧范围进行编辑
- 红色指针表示当前帧位置
- 蓝色背景表示选中的编辑区域

播放控制:

- **播放** - 从当前帧开始播放动作
- **暂停** - 暂停播放
- **停止** - 停止播放并回到第一帧
- **预览当前帧** - 在RViz中显示当前帧的机器人姿态
- **拖动时自动预览** - 勾选后拖动滑块时自动预览

帧选择区域:

- **开始帧** - 设置编辑范围的起始帧
- **结束帧** - 设置编辑范围的结束帧

- 开始设为当前帧 - 将当前帧设为开始帧

- 结束设为当前帧 - 将当前帧设为结束帧

- 显示选中帧数和持续时间

关节编辑区域:

- 新增分组 - 创建新的关节编辑组

- 每组包含: 关节编号、调整值、操作按钮

- 应用到选区 - 将调整应用到选中的帧范围

- 预览 - 预览编辑效果 (不保存)

- 撤销预览 - 恢复预览前的状态

- 平滑帧数 - 设置边缘平滑的帧数

可用关节列表:

- 显示动作中包含的所有关节
- 双击关节将其加入当前编辑组
- 支持多选

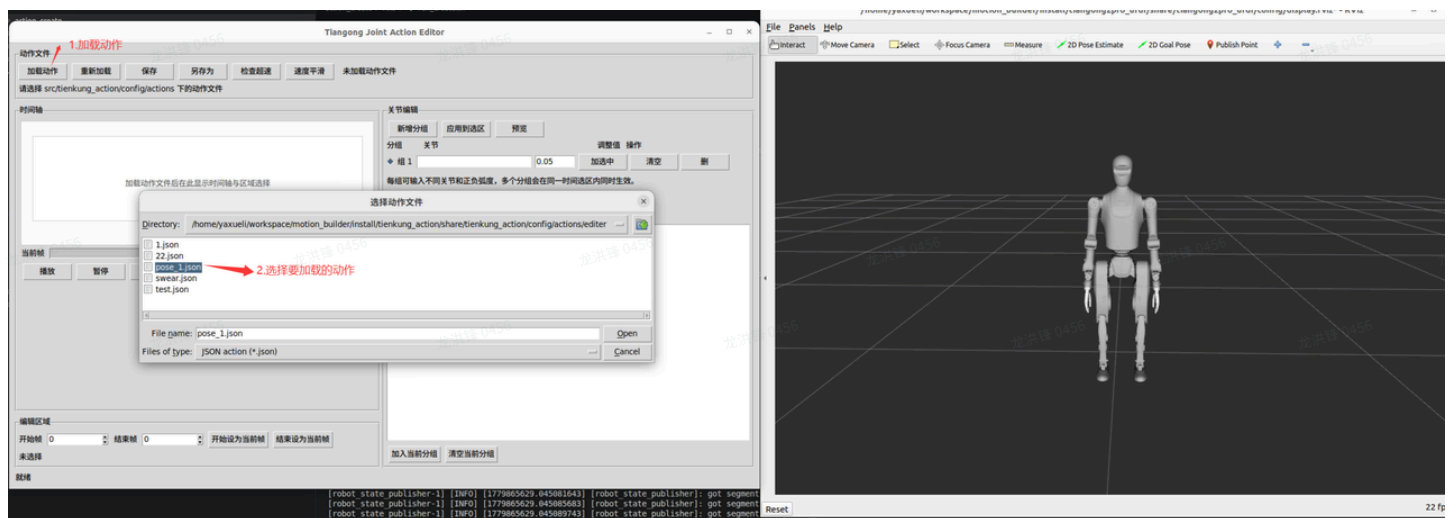
> 注意: 脚本名称与功能存在命名混淆:

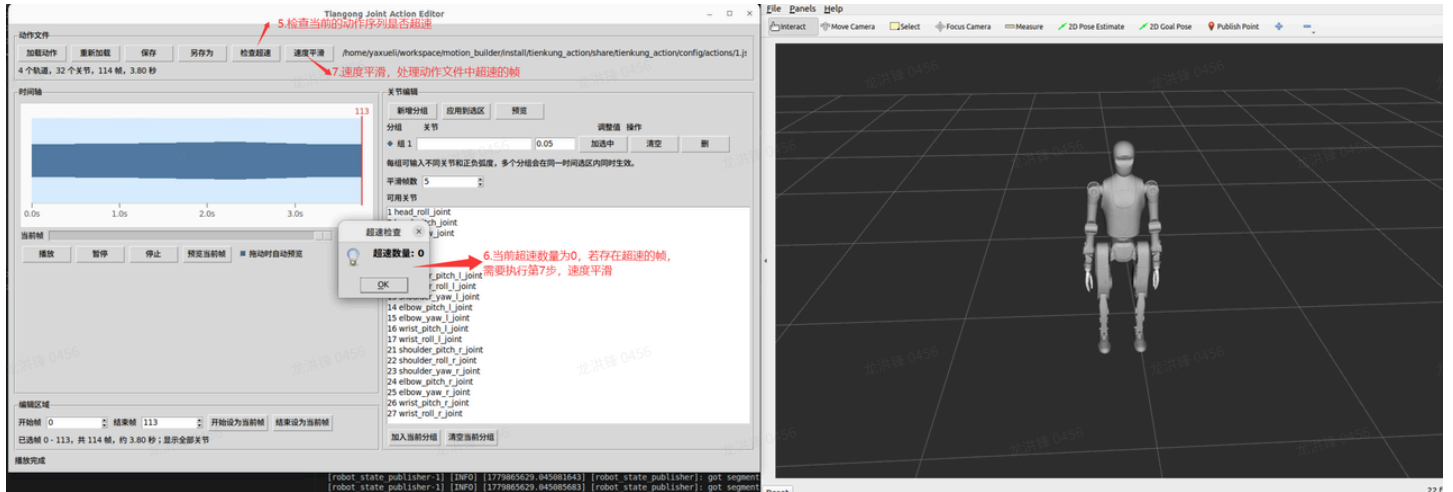
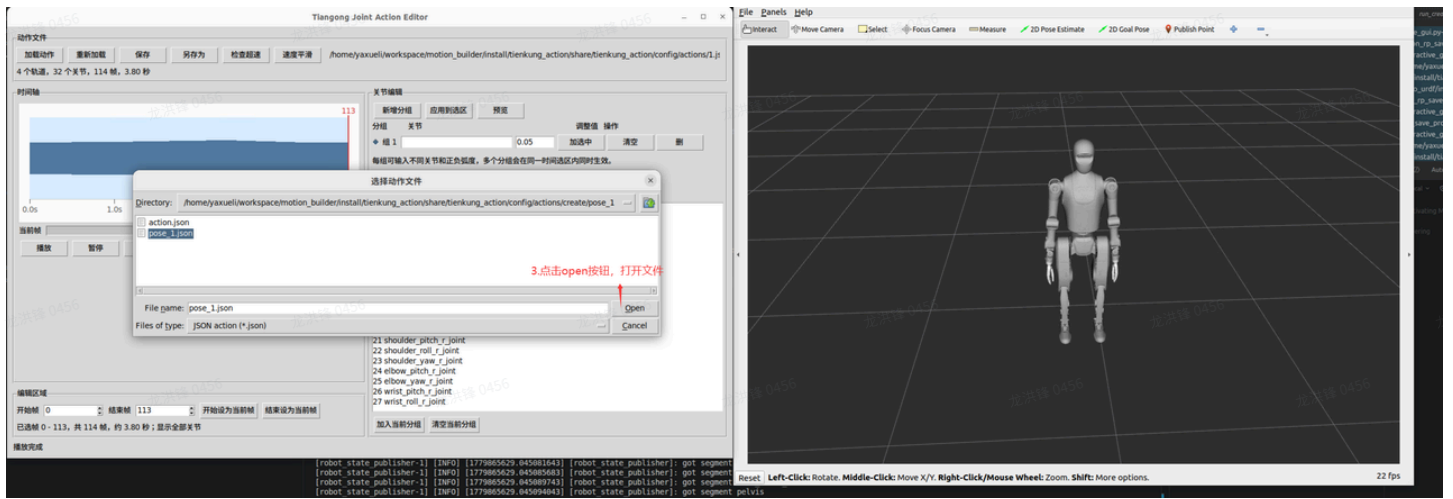
> - `run\_create.sh` 虽然叫"create", 但实际运行的是时间轴编辑器 (用于精修已有动作文件)

> - `run\_editor.sh` 虽然叫"editor", 但实际运行的是Pose创建器 (用于超速检查, 速度平滑, 通过关节数值创建实现矫正动作)

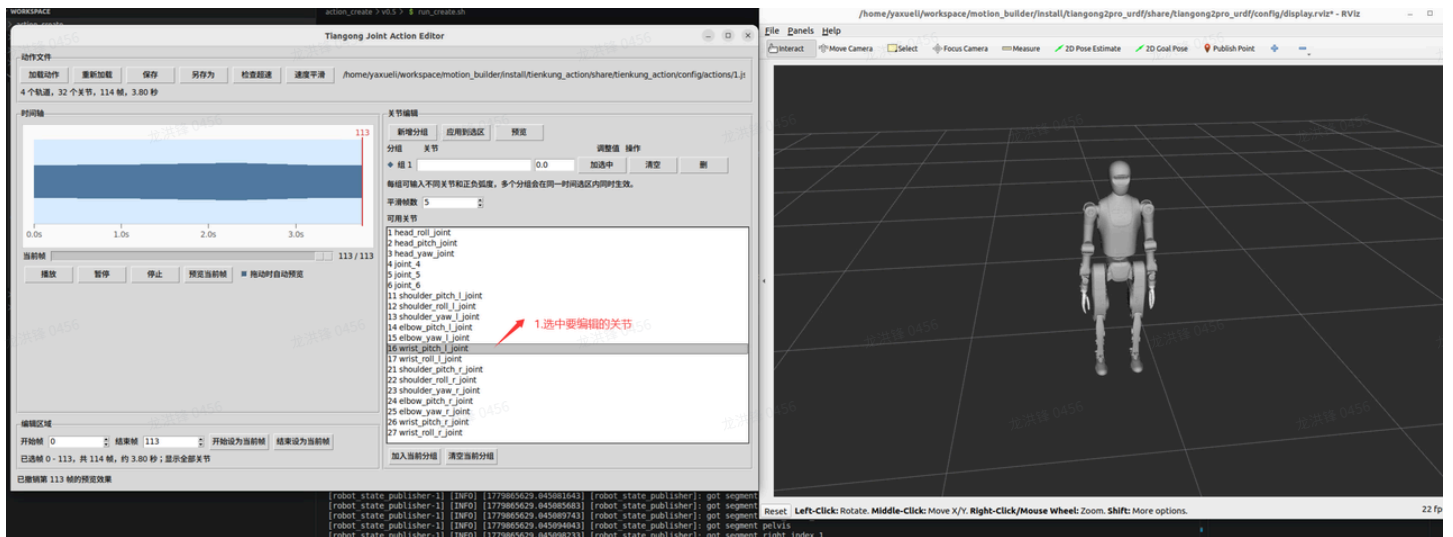
- 使用时请按实际功能选择, 不要被脚本名称误导

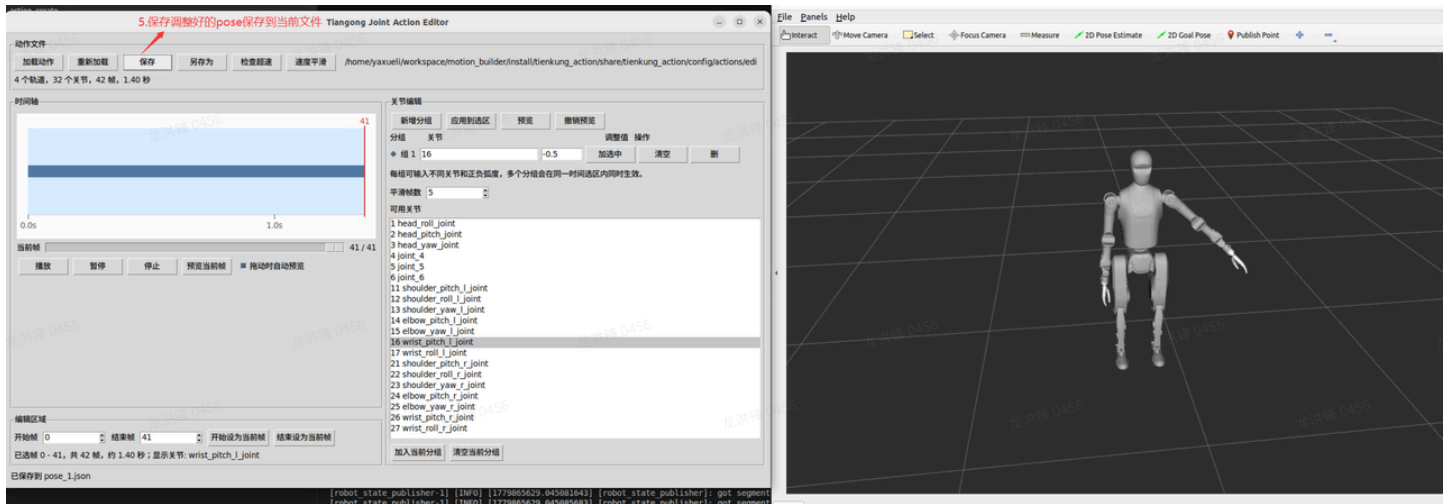
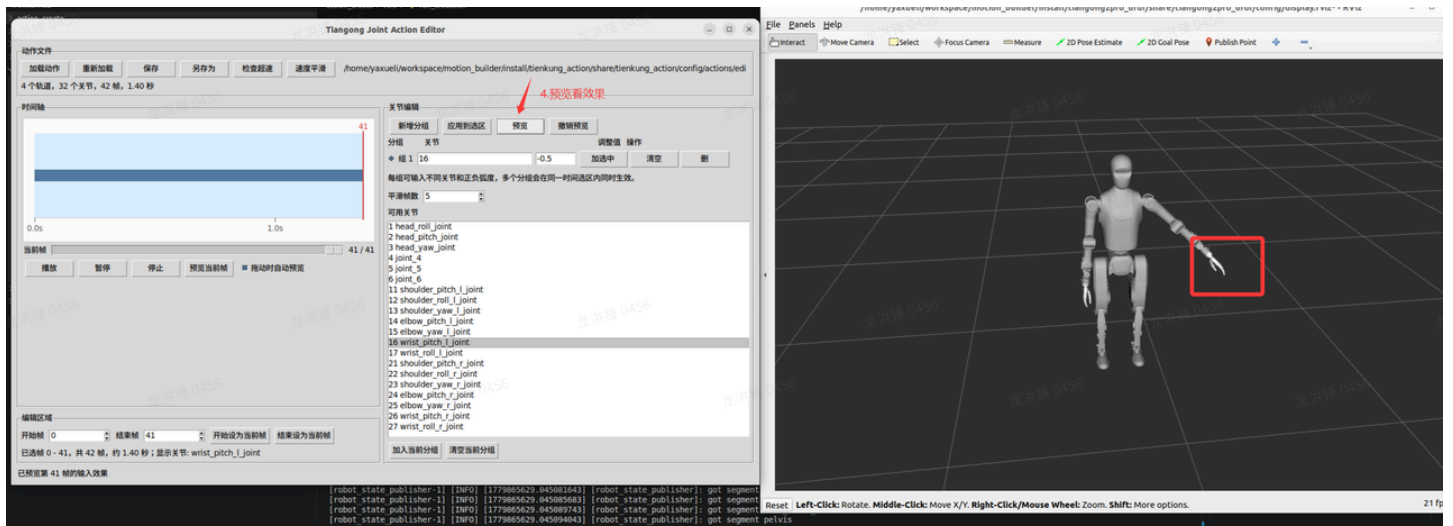
1. 加载动作





2. 编辑动作





4.3 ./run_sim.sh - 仿真实播环境

功能: 在仿真环境中完整播放动作序列, 包含音频和动作。

启动内容:

- `robot_state_publisher` - 机器人状态发布
- `sim_joint_bridge` - 仿真关节桥接
- `audio_service` - 音频播放服务
- `sim_joy` - 遥控器事件监听 (节点名)
- `trigger_player` - 动作播放节点
- `rviz2` - 3D可视化界面

使用的配置文件:

代码块

```
1 src/tienkung_action/config/scenarios.json
```

音频文件位置:

代码块

```
1  # 真机音频地址设定
2  "audio_base_dir": "/home/nvidia/{voice_project}"
3
4  # 本地音频地址设定
5  "sim_base_dir": "src/tienkung_action/data/{voice_project}",
```

动作文件目录:

代码块

```
1  install/tienkung_action/share/tienkung_action/config/actions/
```

配置文件格式说明 (scenarios.json routines):

在 `scenarios.json` 的 `routines` 数组中, 每个条目可以有以下三种配置方式:

1. 只有声音 (voice_only):

代码块

```
1  {
2    "name": "voice_only",
3    "audio": "1.mp3",
4    "audio_duration_sec": 29.0
5  }
```

- `audio`: 音频文件名 (文件需放在 `/home/nvidia` 目录)
- `audio_duration_sec`: 音频播放时长 (秒), 根据MP3文件实际时长设定

2. 声音和动作同时执行:

代码块

```
1  {
2    "name": "动作文件名1",
3    "audio": "3.mp3",
4    "audio_duration_sec": 2.0,
5    "action_file": "actions/动作文件名1.json",
6    "action_start_offset_sec": 0.0,
7    "post_wait_sec": 1.0
8  }
```

- `audio`: 音频文件名
- `audio_duration_sec`: 音频播放时长
- `action_file`: 动作文件路径（相对于 `actions/` 目录）
- `action_start_offset_sec`: 动作开始偏移时间
- `post_wait_sec`: 动作结束后等待时间

3. 只有动作:

代码块

```
1  {
2    "name": "动作文件名2",
3    "action_file": "actions/动作文件名2.json",
4    "action_start_offset_sec": 0.0,
5    "post_wait_sec": 1.0
6  }
```

- 仅执行动作，不播放音频

控制流程说明:

前置准备: 确保 scenarios.json 中配置正确:

- `button_e`: -1 (E键抬起)
- `button_f`: 1 (F键按下)

操作步骤:

步骤	操作	说明
1	将F模式设置为down	按下F键，button_f = 1
2	长按A键	机器人进入站立姿态

3	将E模式设置为up	抬起E键，button_e = -1
4	准备完成	等待触发
5	按B键，再按A键	开始播放第一个动作（如抬手握拳）
6	再次按B键	切换到下一个动作（如播放mp3）
7	再次按B键	切换到下一个动作（如手落下）
8	重复B+A	进入下一轮动作序列

控制逻辑:

- **B键** - 预备(arm)，使能触发器，1秒内有效
- **A键** - 执行(fire)，播放当前选中的动作
- **B+A为一轮** - 按B预备，再按A执行当前动作
- **按B切换下一个** - 一个动作播放完后，按B键切换到预设的下一个动作，再次按A播放

执行模式 (由 scenarios.json 中的 `mode` 字段决定):

模式	配置	说明
Step	<code>"mode": "step"</code>	每次B+A执行一个routine，按B切换到下一个，适合调试
Batch	<code>"mode": "batch"</code>	一次B+A执行全部routines，适合完整表演

4.4 ./run_real.sh - 真实机播放环境

功能: 在真实机器人上播放动作序列（不启动仿真环境，只运行 trigger_player 节点）。

部署步骤

一、上传文件包

1. 上传 MP3 音频文件到 nvidia

代码块

```
1 # 上传 mp3 文件
2 scp ./user_voice.zip nvidia@192.168.41.2:~/
```

2. 上传文件包到 nvidia 控制板

代码块

```
1 # 将 motion_builder 项目打包并上传
2 scp ./motion_builder.zip nvidia@192.168.41.2:~
```

> 注意：根据具体的 MP3 文件时间，在 scenarios.json 中设定对应的 `audio\_duration\_sec` 参数

二、连接机器人并编译

1. SSH 连接到机器人 nvidia 板

代码块

```
1 ssh nvidia@192.168.41.2
2
3 # 启动语音服务（方式一：直接启动）
4 ros2 launch lyre chat.launch.py
5
6 # 启动语音服务（方式二：systemctl服务）
7 sudo systemctl start lyre.service
8
9 # 查看是否开启了语音服务
10 systemctl status lyre.service
11
12 # 重启语音服务
13 sudo systemctl restart lyre.service
```

2. 解压并编译

代码块

```
1 # 解压文件包
2 unzip user_voice.zip
3
4 # 解压 motion_builder
5 unzip motion_builder.zip
6
```

```
7 # 进入项目目录
8 cd motion_builder
9
10 # 清理旧的编译文件
11 rm -rf build install log
12
13 # 编译项目
14 colcon build
```

三、运行真机操作流程

1. 机器人准备与站立

 **重要安全提示：**按下 D 键前，机器人必须固定在保护支架上！

步骤	操作	说明
1	按遥控器上的 D 键	机器人回零到初始状态
2	控制保护支架缓慢下降	将机器人放置到地面
3	保持机器人竖直状态	保持 60 秒 稳定
4	确认 H 拨杆 位置	必须处于初始的中间零位
5	长按 A 键	使机器人站立
6	观察机器人状态	检查是否站立平衡（无抖动、无前后倾倒）

如不平衡，重复以上操作再次尝试。

2. 启动播放程序

代码块

```
1 ./run_real.sh
```

3. 播放动作流程

步骤	操作	说明
1	将 F 模式 设置为 down	按下 F 键，button_f = 1
2	长按 A 键	机器人进入站立
3	将 E 模式 设置为 up	抬起 E 键，button_e = -1
4	长按 A 键	准备完成
5	按 B 键 ，再按 A 键	开始播放动作或语音
6	再次按 B 键	切换下一个动作（或播放誓言 mp3 文件）

再次演示：

- 只需再次按 **B 键**，再按 **A 键**，即可重新开始播放动作序列

四、关闭机器人

关闭机器人时，请严格遵循以下步骤：

步骤	操作	现象/说明
1	确认机器人已停止并返回站立状态	确保动作已完成

- | 2 | 按下遥控器上的 **C 键** | 使机器人僵停 |
- | 3 | 将机器人固定在支架上，并向上吊起 | 确保机器人安全悬空 |
- | 4 | 按下急停按钮 | 开/关机键指示灯：变为 **红色** |
- | 5 | 长按开/关机键 **6 秒** | 开/关机键指示灯：变为 **蓝色** |
- | 6 | 按下总开关键 | 此时所有指示灯全部熄灭 |
- | 7 | 遥控器电源键先短按后长按 | 关闭遥控器 |

五、文件保存位置汇总

工具/脚本	保存位置	说明
-------	------	----

run_create.sh (Pose创建器)	install/tienkung_action/share/tienkung_action/config/actions/create/	通过关节数值创建的新动作
run_editor.sh (动作精修工具)	install/tienkung_action/share/tienkung_action/config/actions/editor/	精修后的动作文件
scenarios.json (运行时)	src/tienkung_action/config/scenarios.json	仿真和真机运行时使用的配置文件
scenarios.json (运行时)	install/tienkung_action/share/tienkung_action/config/actions/	仿真和真机运行时使用的动作文件

六、配置文件说明

6.1 scenarios.json 结构

代码块

```
1  {
2      "mode": "step",                // 执行模式: "step" 或 "batch"
3      "trigger": {
4          "required_states": {
5              "button_e": -1,        // E键必须抬起
6              "button_f": 1         // F键必须按下
7          },
8          "arm_event": "KEY_B_DOWN", // B键按下为预备
9          "fire_event": "KEY_A_DOWN", // A键按下为执行
10         "window_sec": 1.0         // 预备有效期 (秒)
11     },
12     "sbus_topic": "/sbus_data/event", // 遥控器话题
13     "audio_base_dir": "/home/nvidia/traffic_voice", // 音频文件目录 (真实机)
14     "sim_base_dir": "src/tienkung_action/data/traffic_voice", // 音频文件目录 (仿真)
15     "routines": [                // 动作序列
16         {
17             "name": "stop",        // 动作名称
18             "audio": "stop.mp3",
19             "audio_duration_sec": 2.0,
20             "action_file": "actions/editor/stop.json",
21             "action_start_offset_sec": 0.0,
22             "post_wait_sec": 1.0
23         },
```

```

24     {
25         "name": "pass",
26         "audio": "pass.mp3",
27         "audio_duration_sec": 2.0,
28         "action_file": "actions/editer/pass.json",
29         "action_start_offset_sec": 0.0,
30         "post_wait_sec": 1.0
31     }
32 ]
33 }

```

6.2 动作JSON文件结构

代码块

```

1  {
2      "actions": [
3          {
4              "topic": "/arm/cmd_pos",           // 控制话题
5              "message_type": "bodyctrl_msgs/msg/CmdSetMotorPosition",
6              "opts": {
7                  "spd": 2.0,                   // 速度限制
8                  "cur": 8.0                     // 电流限制
9              },
10             "data": {
11                 "join_id": [11, 12, 13],       // 关节ID列表
12                 "keys": [                     // 30Hz关键帧序列
13                     [-0.14, 0.11, -0.02],     // 第1帧
14                     [-0.10, 0.11, -0.08]     // 第2帧
15                 ]
16             }
17         }
18     ]
19 }

```

路径配置说明:

- `audio_base_dir`: 真实机器人上的音频文件路径, 默认为 `/home/nvidia`
- `sim_base_dir`: 仿真环境下声音文件路径, 已配置为相对路径
- `action_file`: 动作文件路径, 相对于 `scenarios.json` 所在目录

> **注意:** 如果需要在其他系统上使用, 请根据实际情况修改 `audio\_base\_dir` 路径。

七、常用命令

7.1 编译

代码块

```
1 # 只编译动作包
2 colcon build --packages-select tienkung_action
3
4 # 编译仿真相关包
5 colcon build --packages-select tiangong2pro_urdf sim_joy tienkung_action
6
7 # 清理后重新编译
8 colcon build --cmake-clean-cache --packages-select tienkung_action
```

7.2 手动启动节点

代码块

```
1 # Source环境
2 source /opt/ros/humble/setup.bash
3 source other_install/lyre_msgs/share/lyre_msgs/local_setup.bash
4 source other_install/bodyctrl_msgs/share/bodyctrl_msgs/local_setup.bash
5 source install/setup.bash
6
7 # 启动播放器
8 ros2 run tienkung_action trigger_player
```

7.3 调试话题

代码块

```
1 # 查看遥控器事件
2 ros2 topic echo /sbus_data/event
3
4 # 查看所有话题
5 ros2 topic list
```



```
6
7 # 查看话题信息
8 ros2 topic info /arm/cmd_pos
```

八、故障排查

8.1 FileNotFoundError: Action file not found

原因: scenarios.json中引用的动作文件路径不正确。

解决方案:

1. 检查 scenarios.json 中的 action_file 路径

2. 确保文件存在于

```
install/tienkung_action/share/tienkung_action/config/actions/
```

3. 如使用 editor/ 子目录, 路径应为 actions/editor/xxx.json

8.2 编译错误

问题: `PackageNotFoundError`

解决方案:

代码块

```
1 # 确保已source ROS 2环境
2 source /opt/ros/humble/setup.bash
3
4 # 确保已source依赖消息包
5 source other_install/lyre_msgs/share/lyre_msgs/local_setup.bash
6 source other_install/bodyctrl_msgs/share/bodyctrl_msgs/local_setup.bash
7
8 # 重新编译
9 colcon build --packages-select tienkung_action
```

8.3 动作没有响应

检查项:

- 遥控器事件是否正常: `ros2 topic echo /sbus_data/event`

- `required_states` 是否满足（E键up，F键down）
- 是否先按B键预备，再按A键执行
- 预备时间是否超过1秒

8.4 有动作但没声音

检查项:

- `audio_service` 是否正常运行（仅run_sim.sh启动）
- `audio_base_dir` 路径是否正确
- 音频文件是否存在

九、注意事项

1. **路径配置:** 修改源文件后必须重新编译才能生效
2. **文件权限:** 确保所有`.sh`脚本有执行权限
3. **环境变量:** 每次新开终端需要source环境，或添加到`~/.bashrc`
4. **动作文件格式:** 必须严格遵循JSON格式，关节ID和角度数组长度要匹配
5. **速度限制:** 动作编辑器中的`spd`参数控制关节运动速度，避免超速



traffic_voice.zip

58.54KB





motion_builder.zip

96.82MB

